జావా టైప్ సిస్టమ్ మరియు జెనెరిక్ టైప్ క్రియేషన్ మాస్టర్‌క్లాస్

సిద్ధాంతపరమైన ఫార్మలైజేషన్, JVM రన్‌టైమ్ మెకానిక్స్, బైట్‌కోడ్ ఎరేజర్ మరియు ఎంటర్‌ప్రైజ్ ఆర్కిటెక్చర్ గైడ్

సారాంశం (ABSTRACT)

ఈ పరిశోధనా పత్రం ఆధునిక జావా (Java 5 నుండి LTS 17, 21 వరకు) టైప్ సిస్టమ్ మరియు జెనెరిక్స్ యొక్క సంపూర్ణ అంతర్గత నిర్మాణాన్ని అత్యంత లోతుగా విశదీకరిస్తుంది. (1) ప్రిమిటివ్ vs రిఫరెన్స్ టైప్స్ మరియు బాక్సింగ్ కాష్ మెకానిక్స్; (2) సబ్‌టైపింగ్ లాటిస్ మరియు ఎరే కోవేరియన్స్ యొక్క ఘోరమైన లోపం (ArrayStoreException); (3) జెనెరిక్ ఇన్వేరియన్స్ యొక్క గణిత నిరూపణ; (4) రికర్సివ్ F-Bounded పాలిమార్ఫిజం (CRTP Fluent Builders); (5) యూజ్-సైట్ వేరియన్స్ మరియు Josh Bloch PECS ప్రిన్సిపల్; (6) టైప్ ఎరేజర్ మరియు సింథటిక్ బ్రిడ్జ్ మెథడ్స్ (ACC_BRIDGE); (7) రీఫికేషన్ పరిమితులు, హీప్ పొల్యూషన్ మరియు @SafeVarargs కాంట్రాక్ట్; (8) సూపర్ టైప్ టోకెన్లు (Neal Gafter Hack) మరియు సీల్డ్ ADTలు; (9) సమగ్ర ప్రొడక్షన్ రిఫరెన్స్ ఆర్కిటెక్చర్లు ఇందులో పొందుపరచబడ్డాయి.

విభాగం 1: జావా టైప్ సిస్టమ్ పునాదులు (Primitive vs Reference Duality)

జావా లాంగ్వేజ్ స్పెసిఫికేషన్ (JLS §4) ప్రకారం జావాలోని అన్ని డేటా టైపులను రెండు విభాగాలుగా విభజించవచ్చు: **Primitive Types** (ప్రాథమిక విలువలు) మరియు **Reference Types** (రిఫరెన్స్ ఆబ్జెక్టులు). హార్డ్‌వేర్ స్థాయి మెమరీ మరియు రిజిస్టర్ ఆప్టిమైజేషన్ కోసం ప్రిమిటివ్స్, ఆబ్జెక్ట్ ఓరియెంటెడ్ పాలిమార్ఫిజం కోసం రిఫరెన్స్ టైప్స్ రూపొందించబడ్డాయి.

1.1 ప్రిమిటివ్ టైప్స్: ఫిజికల్ మెమరీ మరియు బిట్ లేఅవుట్

ప్రిమిటివ్స్ నేరుగా థ్రెడ్ ఎగ్జిక్యూషన్ స్టాక్ (Stack Frame) లేదా ఆబ్జెక్ట్ ఇన్‌లైన్ మెమరీలో స్టోర్ అవుతాయి. వీటికి ప్యూర్ వ్యాల్యూ సెమాంటిక్స్ ఉంటాయి (అసైన్‌మెంట్ చేసినప్పుడు బిట్స్ కాపీ అవుతాయి, పాయింటర్లు కావు).

ప్రిమిటివ్ టైప్	వర్గం	బిట్ వెడల్పు	డిఫాల్ట్ విలువ	విలువల పరిధి (Range)	JVM బైట్‌కోడ్ రిప్రజెంటేషన్
`byte`	Signed Integer	8 bits	`(byte) 0`	-128 నుండి 127 వరకు	స్టాక్‌లో 32-bit `int` గా ఎవాల్యుయేట్ అవుతుంది (`iload`, `istore`)
`short`	Signed Integer	16 bits	`(short) 0`	-32,768 నుండి 32,767 వరకు	స్టాక్‌లో 32-bit `int` గా ఎవాల్యుయేట్ అవుతుంది
`int`	Signed Integer	32 bits	`0`	-2^{31} నుండి $2^{31}-1$ (~2.14 బిలియన్లు)	నేటివ్ 32-bit ఇంటిజర్ (`iload`, `iadd`)
`long`	Signed Integer	64 bits	`0L`	-2^{63} నుండి $2^{63}-1$	నేటివ్ 64-bit పూర్ణాంకం (స్టాక్‌లో 2 స్లాట్లు ఆక్రమిస్తుంది)
`float`	IEEE 754 Floating	32 bits	`0.0f`	1 sign, 8 exponent, 23 mantissa bits	నేటివ్ సింగిల్ ప్రెసిషన్ ఫ్లోట్ (`fload`, `fadd`)
`double`	IEEE 754 Floating	64 bits	`0.0d`	1 sign, 11 exponent, 52 mantissa bits	నేటివ్ డబుల్ ప్రెసిషన్ (స్టాక్‌లో 2 స్లాట్లు)
`char`	Unsigned Unicode	16 bits	`'\u0000'`	0 నుండి 65,535 వరకు (UTF-16 code units)	అన్‌సైన్డ్ 32-bit `int` గా స్టాక్‌లో నడుస్తుంది
`boolean`	Logical Truth	JVM-spec	`false`	{ `false`, `true` }	స్టాక్‌లో 0 లేదా 1 గా, హీప్ ఎరేస్‌లో బైట్‌లుగా నిల్వ అవుతుంది

1.2 న్యూమరిక్ ప్రమోషన్ రూల్స్ (Binary Numeric Promotion)

జావాలో ఏదైనా బైనరీ అంకగణిత చర్య (+, -, *, /, %) జరిగినప్పుడు కింది రూల్స్ వర్తిస్తాయి:

- ఒక ఆపరాండ్ `double` అయితే, రెండవది కూడా `double` గా మారుతుంది.
- ఒకటి `float` అయితే, రెండవది `float` గా మారుతుంది.
- ఒకటి `long` అయితే, రెండవది `long` గా మారుతుంది.
- **లేకపోతే, రెండు ఆపరాండ్లు ఖచ్చితంగా `int` గా ప్రమోట్ అవుతాయి!** (రెండు `byte` లను కూడినా ఫలితం `int` అవుతుంది).

1.3 ది నల్ టైప్ ($\perp_{ref}$) మరియు రిఫరెన్స్ టైప్స్

రిఫరెన్స్ టైప్స్ హీప్ (Heap) లోని ఆబ్జెక్టులను సూచించే పాయింటర్లు. ప్రతి రిఫరెన్స్ ఆబ్జెక్ట్ హెడర్‌లో Mark Word (లాకింగ్, హాష్‌కోడ్, GC ఏజ్) మరియు Klass Word (మెటాస్పేస్ పాయింటర్) ఉంటాయి.

నిర్వచనం: ది నల్ టైప్ సిద్ధాంతం

`null` అనేది anonymous Null Type ($\bot_{ref}$) యొక్క ఏకైక విలువ. ఇది జావాలోని **అన్ని రిఫరెన్స్ టైపులకు direct sub-type** అవుతుంది:

$$\forall\ T \in \text{ReferenceTypes} \Rightarrow \bot_{ref} \leq T$$

కానీ, `null` ఏ ప్రిమిటివ్ టైపుకూ సబ్‌టైప్ కాదు. అందుకే ప్రిమిటివ్‌కు అసైన్ చేస్తే కంపైల్ ఎర్రర్, అన్‌బాక్సింగ్ చేస్తే రన్‌టైమ్‌లో `NullPointerException` వస్తుంది.

1.4 ఆటోబాక్సింగ్ మరియు రాపర్ కాష్ ట్రాప్ (Wrapper Cache Trap)

జావా 5 లో ప్రవేశపెట్టిన ఆటోబాక్సింగ్ ద్వారా ప్రిమిటివ్స్ వాటి రాపర్ క్లాస్‌లుగా (ఉదా: `int → Integer`) మారుతాయి. కంపైలర్ `Integer.valueOf(val)` ని సింథసైజ్ చేస్తుంది.

హెచ్చరిక: ఆబ్జెక్ట్ ఐడెంటిటీ (==) ప్రమాదం

JVM పనితీరు కోసం కొన్ని నిర్దిష్ట పరిధులలో ఫ్లైవెయిట్ కాష్ పూల్స్ నిర్వహిస్తుంది:

- `Byte`, `Short`, `Integer`, `Long`: **[-128 నుండి 127]** వరకు కాష్ చేయబడతాయి.
- `Character`: [0 నుండి 127] వరకు.
- `Float`, `Double`: కాష్ ఉండదు.

జావా రాపర్ కాష్ ప్రమాదం

```
Integer a = 127;
Integer b = 127;
System.out.println(a == b); // TRUE: కాష్ పూల్ నుండి ఒకే ఇన్‌స్టాన్స్!

Integer c = 128;
Integer d = 128;
System.out.println(c == d); // FALSE: హీప్‌లో వేర్వేరు ఆబ్జెక్టులు!
System.out.println(c.equals(d)); // TRUE: కంటెంట్ వ్యాల్యూ ఈక్వాలిటీ
```

విభాగం 2: సబ్‌టైపింగ్ సిద్ధాంతం & ఎరే కోవేరియన్స్ లోపం

జావా స్ట్రిక్ట్ నామినల్ సబ్‌టైపింగ్ (Nominal Subtyping) పై ఆధారపడి ఉంటుంది. `extends` లేదా `implements` ద్వారా స్పష్టంగా డిక్లేర్ చేస్తేనే సబ్‌టైపింగ్ రిలేషన్ ఏర్పడుతుంది. `java.lang.Object` అనేది అన్ని రిఫరెన్స్ టైపులకూ యూనివర్సల్ సూపర్-టైప్.

2.1 ఎరే కోవేరియన్స్ (Array Covariance) మరియు రన్‌టైమ్ క్రాష్

జావా 1.0 లో జెనెరిక్స్ లేనందున, `Arrays.sort(Object[])` వంటి యుటిలిటీ మెథడ్స్ పనిచేయడానికి ఎరేస్‌ను కోవేరియంట్‌గా రూపొందించారు:

$$S \leq T \Rightarrow S[] \leq T[]$$

కానీ ఈ డిజైన్ స్టాటిక్ టైప్ సేఫ్టీకి తీవ్రమైన ముప్పు తెచ్చిపెట్టింది:

ARRAYSTOREEXCEPTION రన్‌టైమ్ వైఫల్యం

```
String[] strArr = new String[5];
Object[] objArr = strArr; // కోవేరియన్స్ వల్ల కంపైల్ అవుతుంది!

// కంపైలర్ దృష్టిలో objArr ఒక Object[]; Integer అనేది Object కి సబ్‌టైప్:
objArr[0] = Integer.valueOf(42);
// FATAL ERROR: రన్‌టైమ్‌లో JVM "java.lang.ArrayStoreException" విసురుతుంది!
```

JVM రక్షణ చర్య: ఎరేస్ *Reified* కావడం వల్ల ప్రతి ఎరే ఆబ్జెక్ట్ తన హెడర్‌లో కాంపోనెంట్ టైప్‌ను భద్రపరుస్తుంది. బైట్‌కోడ్‌లో ఎరే రైట్ (`aastore`) జరిగినప్పుడల్లా JVM ఇన్‌సర్ట్ అయ్యే విలువ ఆ కాంపోనెంట్ టైప్‌కు సరిపోతుందో లేదో రన్‌టైమ్ చెక్ చేస్తుంది. సరిపోకపోతే ప్రోగ్రామ్‌ను ఆపేస్తుంది.

2.2 ప్రిమిటివ్ ఎరేస్: ఇన్వేరియన్స్ రూల్

ఎరే కోవేరియన్స్ కేవలం రిఫరెన్స్ ఎరేస్‌కు మాత్రమే వర్తిస్తుంది. ప్రిమిటివ్ ఎరేస్ ఎట్టి పరిస్థితుల్లోనూ కోవేరియంట్ కావు:

$$\texttt{int[]} \not\leq \texttt{Object[]} \quad \text{మరియు} \quad \texttt{int[]} \not\leq \texttt{Integer[]}$$

`int[]` అనేది నేరుగా `java.lang.Object` కి సబ్‌టైప్ అవుతుంది, కానీ `Object[]` కి కాదు!

విభాగం 3: జెనెరిక్ టైప్ పారామీటరైజేషన్ & ఇన్వేరియన్స్ నిరూపణ

జావా 5 లో రా టైప్స్ (Raw Types) మరియు మ్యాన్యువల్ కాస్టింగ్ వల్ల వచ్చే `ClassCastException` లను నిర్మూలించడానికి జెనెరిక్స్ ప్రవేశపెట్టబడ్డాయి. జెనెరిక్స్ యొక్క అత్యంత ముఖ్యమైన సూత్రం: **Generic Types are Invariant**.

> **థీరమ్: జెనెరిక్ ఇన్వేరియన్స్ సిద్ధాంతం**
>
> ఏవైనా రెండు విభిన్న టైపులు $S \neq T$ మరియు జెనెరిక్ క్లాస్ $C\langle \cdot \rangle$ లకు:
>
> $$S \leq T \not\Rightarrow C<S> \leq C<T>$$
>
> ఉదాహరణకు, `Integer ≤ Number` అయినప్పటికీ, `List<Integer> ≰ List<Number>`.

> **గణిత నిరూపణ (Proof by Heap Contradiction)**
>
> జెనెరిక్స్ కోవేరియంట్ అని విరుద్ధంగా భావిద్దాం: `List<Integer> ≤ List<Number>` అనుకుందాం.
> అప్పుడు కింది కోడ్ చట్టబద్ధం కావాలి:
>
> 1. `List<Integer> intList = new ArrayList<>();`
> 2. `List<Number> numList = intList; // కోవేరియన్స్ వల్ల చెల్లుతుంది`
> 3. `numList.add(Double.valueOf(3.14)); // Double అనేది Number కాబట్టి యాడ్ చేయవచ్చు!`
> 4. `Integer val = intList.get(0); // intList నుండి చదివాం!`
>
> ఇప్పుడు `val` అనే `Integer` వేరియబుల్ మెమరీలో `Double` ఆబ్జెక్టును సూచిస్తుంది! దీనిపై ఇంటిజర్ ఆపరేషన్లు జరిపితే JVM మెమరీ భద్రత ధ్వంసమవుతుంది. కాబట్టి కంపైల్ సమయంలోనే సంపూర్ణ భద్రత కల్పించడానికి, జెనెరిక్ టైప్స్ ఖచ్చితంగా **ఇన్వేరియంట్**గా ఉండాలి.

3.1 స్టాటిక్ ఐసోలేషన్ బౌండరీ (Static Isolation Boundary)

క్లాస్-లెవల్ టైప్ పారామీటర్ T ని స్టాటిక్ ఫీల్డ్స్ లేదా స్టాటిక్ మెథడ్లలో ఉపయోగించలేము. ఎందుకంటే JVM లో క్లాస్ లోడ్ అయినప్పుడు స్టాటిక్ మెంబర్స్ అన్ని ఇన్‌స్టాన్స్‌లకూ ఉమ్మడిగా ఉంటాయి; ఆ సమయంలో ఏ ప్రత్యేకమైన T ఉండదు.

జెనెరిక్ క్లాస్ మరియు స్టాటిక్ ఐసోలేషన్

```
public class Container<T> {
    private T value;

    // COMPILE ERROR: స్టాటిక్ ఫీల్డ్‌లో T ని వాడలేము!
    // public static T globalDefault;

    // చట్టబద్ధమైన స్టాటిక్ జెనెరిక్ మెథడ్: తన స్వంత టైప్ పారామీటర్ U ని డిక్లేర్ చేసుకుంటుంది
    public static <U> Container<U> of(U val) {
        return new Container<>(val);
    }
}
```

విభాగం 4: టైప్ బౌండ్స్ & రికర్సివ్ F-Bounded పాలిమార్ఫిజం (CRTP)

టైప్ పారామీటర్‌పై పరిమితులు విధించడానికి అప్పర్ బౌండ్స్ (`<T extends B>`) ఉపయోగిస్తాము. దీనివల్ల ఆ బౌండ్ క్లాస్‌లోని మెథడ్స్‌ను నేరుగా కాల్ చేయవచ్చు.

4.1 మల్టిపుల్ ఇంటర్‌సెక్షన్ బౌండ్స్ (Multiple Intersection Bounds)

జావా & ఆపరేటర్ ద్వారా ఒకేసారి క్లాస్ మరియు పలు ఇంటర్‌ఫేస్‌లను బౌండ్‌గా అనుమతిస్తుంది. JLS §4.4 ప్రకారం:

- గరిష్ఠంగా **ఒకే ఒక క్లాస్ బౌండ్** మాత్రమే ఉండాలి.
- క్లాస్ బౌండ్ ఉంటే, అది **ఖచ్చితంగా మొదటి స్థానంలోనే** ఉండాలి!
- దాని తర్వాత ఎన్ని ఇంటర్‌ఫేస్ బౌండ్స్ అయినా ఉండవచ్చు.

ఇంటర్‌సెక్షన్ బౌండ్స్ రూల్స్

```
// LEGAL: క్లాస్ మొదట వచ్చింది, తర్వాత ఇంటర్‌ఫేస్‌లు వచ్చాయి
public <T extends Number & Comparable<T> & Serializable> void process(T val) { ... }

// COMPILE ERROR: ఇంటర్‌ఫేస్ క్లాస్ కంటే ముందు రాకూడదు!
// public <T extends Comparable<T> & Number> void invalid(T val) { ... }
```

4.2 రికర్సివ్ F-Bounded పాలిమార్ఫిజం (CRTP Fluent Builder)

ఒక ఇంటర్‌ఫేస్ లేదా క్లాస్ తనను ఇంప్లిమెంట్ చేసే కాంక్రీట్ సబ్‌టైప్‌నే పారామీటర్‌గా తీసుకోవడాన్ని F-Bounded పాలిమార్ఫిజం అంటారు (ఉదా: `Comparable<T>`). మల్టీ-లెవల్ బిల్డర్లలో చైనింగ్ బ్రేక్ కాకుండా ఉండటానికి ఇది అత్యద్భుతంగా ఉపయోగపడుతుంది.

CRTP సెల్ఫ్-టైప్డ్ హైరార్కికల్ బిల్డర్

```
// బేస్ బిల్డర్ తన కాంక్రీట్ సబ్‌టైప్ T పై డిపెండ్ అవుతుంది
public abstract class EntityBuilder<T extends EntityBuilder<T>> {
    protected String id;

    @SuppressWarnings("unchecked")
    protected T self() {
        return (T) this; // కాంక్రీట్ సబ్‌టైప్‌కి సురక్షితమైన కాస్ట్
    }

    public T withId(String id) {
        this.id = id;
        return self(); // సబ్‌క్లాస్ రిటర్న్ అవుతుంది, చైనింగ్ బ్రేక్ అవ్వదు!
    }
}

public class UserBuilder extends EntityBuilder<UserBuilder> {
    private String email;

    public UserBuilder withEmail(String email) {
        this.email = email;
        return self();
    }

    public User build() { return new User(id, email); }
}

// క్లయింట్ కోడ్: ఎటువంటి మ్యాన్యువల్ కాస్టింగ్ లేకుండా చైన్ సాగుతుంది!
User user = new UserBuilder()
    .withId("USR_101")
    .withEmail("dev@enterprise.io")
    .build();
```

విభాగం 5: వైల్డ్‌కార్డ్స్ & Josh Bloch PECS ప్రిన్సిపల్

జెనెరిక్ ఇన్వేరియన్స్ వల్ల వచ్చే రిజిడిటీని తగ్గించడానికి జావా యూజ్-సైట్ వేరియన్స్ (Use-Site Variance) గా **Wildcards** ను ప్రవేశపెట్టింది.

డిక్లరేషన్	వేరియన్స్ మోడల్	రీడ్ ఆపరేషన్ (`get()`)	రైట్ ఆపరేషన్ (`add()`)	పాత్ర (Role)
`List<T>`	Invariant	ఖచ్చితంగా T రిటర్న్ అవుతుంది	T మరియు దాని సబ్‌టైప్స్ అనుమతించబడతాయి	Read / Write Container
`List<? extends T>`	**Covariant**	ఖచ్చితంగా T రిటర్న్ అవుతుంది	**ఏదీ అనుమతించబడదు** (కేవలం `null` తప్ప)	**Producer (Read-Only)**
`List<? super T>`	**Contravariant**	కేవలం `Object` మాత్రమే వస్తుంది	T మరియు దాని సబ్‌టైప్స్ అనుమతించబడతాయి	**Consumer (Write-Only)**
`List<?>`	Bivariant	కేవలం `Object` వస్తుంది	ఏదీ అనుమతించబడదు (కేవలం `null` తప్ప)	Unknown Inspection

> **ది PECS ప్రిన్సిపల్ (Producer Extends, Consumer Super)**
>
> - ఒక పారామీటర్ నుండి డేటాను **చదవడానికి (Read)** మాత్రమే ఉపయోగిస్తే, అది Producer ⇒ `? extends T` వాడాలి.
> - ఒక పారామీటర్‌లోకి డేటాను **రాయడానికి (Write)** మాత్రమే ఉపయోగిస్తే, అది Consumer ⇒ `? super T` వాడాలి.
> - రెండు పనులూ చేయాల్సి వస్తే వైల్డ్‌కార్డ్స్ వాడకూడదు; ఖచ్చితమైన ఇన్వేరియంట్ T వాడాలి.

COLLECTIONS.COPY లో PECS క్లాసిక్ ఉదాహరణ

```
public static <T> void copy(List<? super T> dest, List<? extends T> src) {
    for (int i = 0; i < src.size(); i++) {
        T elem = src.get(i); // src Producer: చదివినప్పుడు T గ్యారెంటీ!
        dest.set(i, elem);   // dest Consumer: T ని భద్రంగా రైట్ చేయవచ్చు!
    }
}
```

విభాగం 6: టైప్ ఎరేజర్ & సింథటిక్ బ్రిడ్జ్ మెథడ్స్ (JVM బైట్‌కోడ్)

జావా జెనెరిక్స్ **Compilation by Type Erasure** ద్వారా పనిచేస్తాయి. అంటే కంపైల్ అయిన తర్వాత `.class` ఫైల్‌లో జెనెరిక్ టైప్స్ తొలగించబడతాయి (అన్‌బౌండెడ్ `Object` గా, బౌండెడ్ మొదటి బౌండ్‌గా మారుతాయి). దీనివల్ల పాత JVM వెర్షన్లతో పూర్తి బైనరీ బ్యాక్‌వర్డ్ అనుకూలత లభిస్తుంది.

6.1 సింథటిక్ బ్రిడ్జ్ మెథడ్ ఎందుకు అవసరం? (ACC_BRIDGE)

సబ్‌క్లాస్ ఒక జెనెరిక్ ఇంటర్‌ఫేస్‌ను కాంక్రీట్ టైప్‌తో ఓవర్‌రైడ్ చేసినప్పుడు, బైట్‌కోడ్ లెవెల్‌లో సిగ్నేచర్ మిస్‌మ్యాచ్ ఏర్పడుతుంది:

బ్రిడ్జ్ మెథడ్ ఏర్పడే విధానం

```java
public interface Processor<T> {
    void process(T data); // ఎరేజర్ తర్వాత: void process(Object data);
}

public class StringProcessor implements Processor<String> {
    @Override
    public void process(String data) { ... } // బైట్‌కోడ్: (Ljava/lang/String;)V
}
```

JVM దృష్టిలో (Object)V మరియు (String)V రెండు వేర్వేరు మెథడ్స్! కాబట్టి పాలిమార్ఫిజం పనిచేయడానికి కంపైలర్ ఆటోమేటిక్‌గా ఒక బ్రిడ్జ్ మెథడ్‌ను బైట్‌కోడ్‌లో ఇంజెక్ట్ చేస్తుంది:

కంపైలర్ జనరేట్ చేసే సింథటిక్ బ్రిడ్జ్ మెథడ్ (డికంపైల్డ్)

```java
// Flags: ACC_BRIDGE, ACC_SYNTHETIC
public void process(Object data) {
    this.process((String) data); // కాస్ట్ చేసి ఒరిజినల్ మెథడ్‌కు పంపుతుంది
}
```

విభాగం 7: రీఫికేషన్ పరిమితులు, హీప్ పొల్యూషన్ & @SafeVarargs

7.1 జెనెరిక్ ఎరే క్రియేషన్ ఎందుకు నిషిద్ధం?

జావాలో `new List<String>[10]` లేదా `new T[10]` రాయడం కంపైలర్ నిషేధించింది. ఎందుకంటే ఎరేస్ కోవేరియంట్ మరియు రీఫైడ్; జెనెరిక్స్ ఇన్వేరియంట్ మరియు ఎరేజ్డ్. ఈ రెండూ కలిస్తే రన్‌టైమ్ టైప్ సేఫ్టీ పూర్తిగా కుప్పకూలుతుంది:

జెనెరిక్ ఎరే క్రియేషన్ అనుమతిస్తే జరిగే విపత్తు

```java
// ఇమాజిన్ చేయండి ఇది లీగల్ అయితే:
List<String>[] stringLists = new List<String>[1];
Object[] objects = stringLists; // ఎరే కోవేరియన్స్ వల్ల లీగల్!

List<Integer> intList = List.of(42);
objects[0] = intList; // రన్‌టైమ్‌లో రెండూ ArrayList కాబట్టి ArrayStoreException రాదు!

String s = stringLists[0].get(0); // FATAL ClassCastException!
```

7.2 హీప్ పొల్యూషన్ మరియు @SafeVarargs కాంట్రాక్ట్

జెనెరిక్ వర్గ్‌ఆర్గ్స్ (`T... items`) వాడినప్పుడు అంతర్గతంగా జెనెరిక్ ఎరే క్రియేట్ అవుతుంది, దీనివల్ల హీప్ పొల్యూషన్ వార్నింగ్ వస్తుంది. సదరు మెథడ్ ఆ ఎరేలో కొత్తవి రాయకుండా, ఆ ఎరే రిఫరెన్స్‌ను బయటకు లీక్ చేయకుండా ఉంటే, `@SafeVarargs` తో వార్నింగ్‌ను అణచివేయవచ్చు (ఇది కేవలం `static`, `final`, లేదా `private` మెథడ్స్‌కి మాత్రమే అనుమతించబడుతుంది).

విభాగం 8: అడ్వాన్స్‌డ్ జెనెరిక్ డిజైన్ ప్యాటర్న్స్

8.1 సూపర్ టైప్ టోకెన్లు (Neal Gafter's Super Type Token)

ఎరేజర్ వల్ల సాధారణంగా రన్‌టైమ్‌లో `List<String>.class` సాధ్యం కాదు. కానీ అనామక ఇన్నర్ సబ్‌క్లాస్ డిక్లరేషన్లలో జెనెరిక్ మెటాడేటా క్లాస్ సిగ్నేచర్‌లో భద్రంగా ఉంటుందని నీల్ గాఫ్టర్ నిరూపించారు. Jackson, Gson, Guice, Spring ఈ టెక్నిక్‌నే వాడుతాయి.

సూపర్ టైప్ టోకెన్ ఆర్కిటెక్చర్

```
public abstract class TypeToken<T> {
    private final java.lang.reflect.Type runtimeType;

    protected TypeToken() {
        java.lang.reflect.Type superclass = getClass().getGenericSuperclass();
        var parameterized = (java.lang.reflect.ParameterizedType) superclass;
        this.runtimeType = parameterized.getActualTypeArguments()[0];
    }

    public java.lang.reflect.Type getType() { return runtimeType; }
}

// క్లయింట్ వాడకం:
TypeToken<List<Map<String, Integer>>> token = new TypeToken<>() {};
System.out.println("పూర్తి రన్‌టైమ్ టైప్: " + token.getType());
// అవుట్‌పుట్: java.util.List<java.util.Map<java.lang.String, java.lang.Integer>>
```

8.2 హెటెరోజీనియస్ టైప్-సేఫ్ కంటైనర్ (Heterogeneous Container)

సాధారణ కలెక్షన్లలో కంటైనర్‌కు ఒకే టైప్ పారామీటర్ ఉంటుంది. కానీ ఇక్కడ 'కీ'లు స్వయంగా టైప్ టోకెన్లుగా మారి, వేర్వేరు డేటా టైపులను ఒకే మ్యాప్‌లో 100% టైప్-సేఫ్‌గా నిల్వ చేస్తాయి.

హెటెరోజీనియస్ రిజిస్ట్రీ ఇంప్లిమెంటేషన్

```
public class TypeSafeRegistry {
    private final Map<Class<?>, Object> map = new HashMap<>();

    public <T> void register(Class<T> type, T instance) {
        map.put(Objects.requireNonNull(type), type.cast(instance));
    }

    public <T> T get(Class<T> type) {
        return type.cast(map.get(type));
    }
}
```

విభాగం 9: ఎంటర్‌ప్రైజ్ మాస్టర్ డెసిషన్ మ్యాట్రిక్స్

డిజైన్ అవసరం	సిఫార్సు చేయబడిన సింటాక్స్ / ప్యాటర్న్	ప్రధాన పరిమితి / నియమం	JDK ప్రామాణిక ఉదాహరణ
యూనిఫాం డేటా స్టోరేజ్	Generic Class `class Box<T>`	స్టాటిక్ మెంబర్స్ T ని యాక్సెస్ చేయలేవు	`ArrayList<E>`
Read-Only డేటా ప్రొడ్యూసర్	Upper-Bounded Wildcard `? extends T`	కొత్త విలువలను యాడ్ చేయలేము (కేవలం `null` తప్ప)	`Stream.map()`
Write-Only డేటా కన్స్యూమర్	Lower-Bounded Wildcard `? super T`	బయటకు తీస్తే కేవలం `Object` మాత్రమే వస్తుంది	`Collections.sort(list, comp)`
ఫ్లూయెంట్ బిల్డర్ సబ్‌క్లాసింగ్	CRTP `<B extends Builder<B>>`	`self()` మెథడ్ ఇంప్లిమెంట్ చేయాలి	`ProcessBuilder`
రన్‌టైమ్ టైప్ ప్రెజర్వేషన్	Super Type Token `new TypeToken<T>(){}`	అనామక ఇన్నర్ క్లాస్ ఇన్‌స్టాంటియేషన్ అవసరం	Jackson `TypeReference`
మల్టీ-టైప్ డైనమిక్ రిజిస్ట్రీ	Heterogeneous Container `Map<Class<?>, Object>`	పారామీటరైజ్డ్ టైప్స్‌ని నేరుగా స్టోర్ చేయలేదు	Spring `ApplicationContext.getBean()`

ముగింపు (Conclusion)

జావా టైప్ సిస్టమ్ అనేది కేవలం సింటాక్స్ నియమాల సమాహారం కాదు; అది గణితపరమైన టైప్ థాటిస్ మరియు JVM రన్‌టైమ్ సామర్థ్యాల మధ్య సాధించిన సమతుల్యత. ప్రిమిటివ్స్ మరియు రిఫరెన్స్ టైప్స్ మధ్య వ్యత్యాసాన్ని అర్థం చేసుకోవడం, ఎరే కోవేరియన్స్ యొక్క లోపాలను నివారించడం, PECS ప్రిన్సిపల్‌ను సరిగ్గా అమలు చేయడం, మరియు సూపర్ టైప్ టోకెన్స్ వంటి అధునాతన ప్యాటర్న్స్‌ను ఉపయోగించడం ద్వారా ఎంటర్‌ప్రైజ్ స్థాయి సాఫ్ట్‌వేర్ ఆర్కిటెక్చర్‌ను అత్యంత పటిష్టంగా మరియు లోపరహితంగా నిర్మించవచ్చు.